XQuery

Introducción

De acuerdo al incremento en la cantidad de información que es almacenada, intercambiada y presentada usando XML, la habilidad para consultar fuentes de datos XML llega a ser muy importante. El XML tiene la flexibilidad de representar diferentes tipos de información desde diversas fuentes. Para explotar esta flexibilidad, un lenguaje de consulta XML debe proveer componentes para obtener e interpretar la información desde diversas fuentes.

Qué es XQuery?

* XQuery es el lenguaje para consultar datos XML.
* XQuery para XML es como el SQL para las BD's.
* XQuery fue construido sobre expresiones XPath
* XQuery es soportado por la mayor parte de los proveedores de BD's (IBM, Oracle, Microsoft, etc.)
* XQuery es una recomendación W3C.

¿Por qué necesitamos XQuery?

Actualmente, XML se ha convertido en una herramienta de uso cotidiano. Sin embargo, a medida que se emplea en un mayor número de proyectos de complejidad y tamaño crecientes y la cantidad de datos almacenados en XML aumenta, se comprueba que, las herramientas más habituales para manipular desde un programa un árbol con un conjunto de datos en XML, los parsers SAX y DOM, no son prácticas para manejar grandes y complejas colecciones de datos en XML.

XQuery es un lenguaje funcional, lo que significa que, en vez de ejecutar una lista de comandos como un lenguaje procedimental clásico, cada consulta es una expresión que es evaluada y devuelve un resultado, al igual que en SQL.

Documento XML Ejemplo (libros.xml)

```
<?xml version="1.0" encoding="ISO-8859-1"?>

<bib>
    <libro año="1994">
        <titulo>TCP/IP Illustrated</titulo>
        <autor>
           <apellido>Stevens</apellido>
           <nombre>W.</nombre>
     </autor>
        <editorial>Addison-Wesley</editorial>
        <precio> 65.95</precio>
    </libro>

    <libro año="1992">
        <titulo>Advan Programming for Unix environment</titulo>
        <autor>
```

```
            <apellido>Stevens</apellido>
            <nombre>W.</nombre>
      </autor>
         <editorial>Addison-Wesley</editorial>
         <precio>65.95</precio>
    </libro>

    <libro año="2000">
         <titulo>Data on the Web</titulo>
         <autor>
            <apellido>Abiteboul</apellido>
            <nombre>Serge</nombre>
      </autor>
         <autor>
            <apellido>Buneman</apellido>
            <nombre>Peter</nombre>
      </autor>
         <autor>
            <apellido>Suciu</apellido>
            <nombre>Dan</nombre>
      </autor>
         <editorial>Morgan Kaufmann editorials</editorial>
         <precio>39.95</precio>
    </libro>

    <libro año="1999">
         <titulo> Economics of Technology for Digital TV</titulo>
         <editor>
                  <apellido>Gerbarg</apellido>
                  <nombre>Darcy</nombre>
                  <afiliacion>CITI</afiliacion>
         </editor>
             <editorial>Kluwer Academic editorials</editorial>
         <precio>129.95</precio>
    </libro>

</bib>
```

Documento XML Ejemplo (comentarios.xml)

```
<?xml version="1.0" encoding="ISO-8859-1"?>

<comentarios>
    <entrada>
        <titulo>Data on the Web</titulo>
        <precio>34.95</precio>
        <comentario>
     Un libro muy bueno sobre bases de datos.
        </comentario>
    </entrada>
    <entrada>
        <titulo>Advanced Programming in the Unix
environment</titulo>
        <precio>65.95</precio>
        <comentario>
     Un libro claro y detallado de programación en UNIX.
        </comentario>
    </entrada>
    <entrada>
        <titulo>TCP/IP Illustrated</titulo>
        <precio>65.95</precio>
        <comentario>
     Uno de los mejores libros de TCP/IP
        </comentario>
    </entrada>
</comentarios>
```

Consultas en XQuery

Una consulta en XQuery es una expresión que lee una secuencia de datos en XML y devuelve como resultado otra secuencia de datos en XML.

Un detalle importante es que, a diferencia de lo que sucede en SQL, en XQuery las expresiones y los valores que devuelven son dependientes del contexto. Por ejemplo: los nodos que aparecerán en el resultado dependen de los namespaces, de la posición donde aparezca la etiqueta raíz del nodo (dentro de otra, por ejemplo), etc.

En XQuery las consultas pueden estar compuestas por cláusulas de hasta cinco tipos distintos. Las consultas siguen la norma FLWOR, siendo FLWOR las siglas de For, Let, Where, Order y Return.

Consultas en XQuery

En la siguiente tabla se describe la función de cada bloque:

For	Vincula una o más variables a expresiones escritas en XPath, creando un flujo de tuplas en el que cada tupla está vinculada a una de las variable.
Let	Vincula una variable al resultado completo de una expresión añadiendo esos vínculos a las tuplas generadas por una cláusula for o, si no existe ninguna cláusula for, creando una única tupla que contenga esos vínculos.
Where	Filtra las tuplas eliminando todos los valores que no cumplan las condiciones dadas.
Order by	Ordena las tuplas según el criterio dado.
Return	Construye el resultado de la consulta para una tupla dada, después de haber sido filtrada por la cláusula where y ordenada por la cláusula order by.

Consultas en XQuery

En el siguiente ejemplo de cláusula for, la variable $b tomará como valor cada uno de los nodos libros que contenga el archivo “libros.xml”. Cada uno de esos nodos libros, será una tupla vinculada a la variable $b.

```
for $b in document("libros.xml”)//bib/libro
```

A continuación se muestra un ejemplo de una consulta donde aparecen las 5 clausulas.

```
for $b in doc("libros.xml")//libro
let $c := $b//autor
where count($c) > 2
order by $b/titulo
return $b/ titulo
```

Resultado de la consulta:

```
<title>Data on the Web</title>
```

Consultas en XQuery

Las barras: “//” son parte de la expresión XPath que indica la localización de los valores que tomará la variable $b. En esta consulta la función count() hace la misma función que en SQL, contar el número de elementos, nodos en este caso, referenciados por la variable $c.

Una expresión FLWOR vincula variables a valores con cláusulas for y let y utiliza esos vínculos para crear nuevas estructuras de datos XML.

Consultas en XQuery

A continuación se muestra otro ejemplo de consulta XQuery. La siguiente consulta devuelve los títulos de los libros del año 2.000. Como “año” es un atributo y no un elemento se le antecede con un carácter “@”.

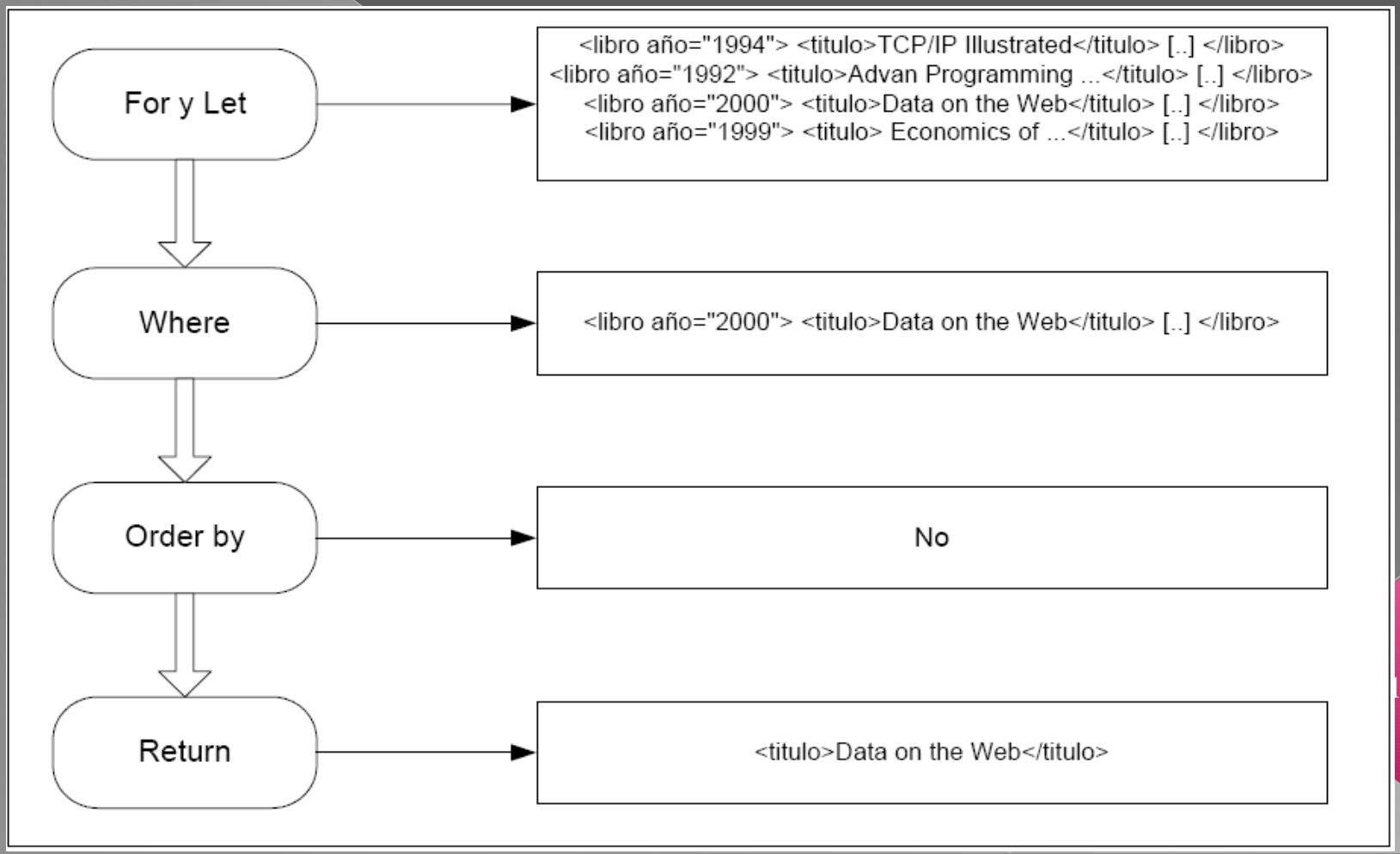
```
for $b in doc("libros.xml")//libro
where $b/@año = "2000"
return $b/titulo
```

Resultado de la consulta:

```
<title>Data on the Web</title>
```

Consultas en XQuery

A continuación se muestran los resultados parciales de la consulta:



Reglas Generales de consultas XQuery

· For y let sirven para crear las tuplas con las que trabajará el resto de las cláusulas de la consulta y pueden usarse tantas veces como se desee en una consulta, incluso dentro de otras cláusulas. Sin embargo solo puede declararse una única cláusula where, una única cláusula order by y una única cláusula return.

· Ninguna de las cláusulas FLWOR es obligatoria en una consulta XQuery. Por ejemplo, una expresión XPath, como la que se muestra a continuación, es una consulta válida y no contiene ninguna de las cláusulas FLWOR.

```
doc("libros.xml")/bib/libro/titulo[/bib/libro/autor/apellido='St
evens']
```

Diferencias entre las cláusulas For y Let

Consulta con For:

```
for $d in doc("libros.xml")/bib/libro/titulo
return
     <titulos>{ $d }</titulos>
```

Resultado de la consulta:

```
<titulos>
     <titulo>TCP/IP Illustrated</titulo>
</titulos>
<titulos>
     <titulo>Advan Programming for Unix environment</titulo>
</titulos>
<titulos>
     <titulo>Data on the Web</titulo>
</titulos>
<titulos>
     <titulo> Economics of Technology for Digital TV</titulo>
</titulos>
```

Diferencias entre las cláusulas For y Let

Consulta con Let:

```
let $d := doc("libros.xml")/bib/libro/titulo
return
<titulos>{ $d }</titulos>
```

Resultado de la consulta:

```
<titulos>
      <titulo>TCP/IP Illustrated</titulo>
      <titulo>Advan Programming for Unix environment</titulo>
      <titulo>Data on the Web</titulo>
      <titulo> Economics of Technology for Digital TV</titulo>
</titulos>
```

Comparando los resultados obtenidos por ambas consultas, la cláusula for vincula una variable con cada nodo que encuentre en la colección de datos. Por este motivo aparece repetido el par de etiquetas <titulos>...</titulos> para cada título. La cláusula let, en cambio, vincula una variable con todo el resultado de una expresión. En este ejemplo, la variable $d se vincula a todos los títulos de todos los libros del archivo "libros.xml", creando una única tupla con todos esos títulos.

Ejemplos de las cláusulas For y Let

Si una cláusula let aparece en una consulta que ya posee una o más cláusulas for, los valores de la variable vinculada por la cláusula let se añaden a cada una de las tuplas generadas por la cláusula for. Un ejemplo se muestra en la siguiente consulta:

```
for $b in doc("libros.xml")//libro
let $c := $b/autor
return
<libro>{ $b/titulo, <autores>{ count($c) }</autores>}</libro>
```

Ejemplos de las cláusulas For y Let

Resultado de la consulta:

```
<libro>
      <titulo>TCP/IP Illustrated</titulo>
      <autores>1</autores>
</libro>
<libro>
      <titulo>Advanced Programming in the UNIX
Environment</titulo>
      <autores>1</autores>
</libro>
<libro>
      <titulo>Data on the Web</titulo>
      <autores>3</autores>
</libro>
<libro>
      <titulo>The Economics of Technology and Content for
Digital TV</titulo>
      <autores>0</autores>
</libro>
```

Ejemplos de las cláusulas For y Let

Si en la consulta aparece más de una cláusula for (o más de una variable en una clásula for), el resultado es el producto cartesiano de dichas variables, es decir, las tuplas generadas cubren todas las posibles combinaciones de los nodos de dichas variables. Un ejemplo se muestra en la siguiente consulta, la cual devuelve los títulos de todos los libros contenidos en el archivo “libros.xml” y todos los comentarios de cada libro contenidos en el archivo “comentarios.xml”.

```
for $t in doc("books.xml")//titulo,
     $e in doc("comentarios.xml")//entrada
where $t = $e/titulo
return <comentario>{ $t, $e/comentario }</comentario>
```

Ejemplos de las cláusulas For y Let

Resultado de la consulta:

```
<comentario>
     <titulo>Data on the Web</titulo>
     <comentario>Un libro muy bueno sobre bases de
datos.</comentario>
</comentario>
<comentario>
     <titulo>Advanced Programming in the Unix
environment</titulo>
     <comentario>Un libro claro y detallado de programación en
UNIX.</comentario>
</comentario>
<comentario>
     <titulo>TCP/IP Illustrated</titulo>
     <comentario>Uno de los mejores libros de
TCP/IP.</comentario>
</comentario>
```

Funciones XQuery:

Xquery ofrece además funciones para la recuperación de información.

Estas funciones son:

Matemáticos:	+, -, *, div(*), idiv(*), mod.
Comparación:	=, !=, <, >, <=, >=, not()
Secuencia:	union (|), intersect, except
Redondeo:	floor(), ceiling(), round().
Funciones de agrupación:	count(), min(), max(), avg(), sum().
Funciones de cadena:	concat(), string-length(), starts-with(), ends-with(), substring(), upper-case(), lower-case(), string()
Uso general:	distinct-values(), empty(), exits()

Aplicación de funciones XQuery

El operador unión recibe dos secuencias de nodos y devuelve una secuencia con todos los nodos existentes en las dos secuencias originales. A continuación se muestra una consulta que usa el operador unión para obtener una lista ordenada de apellidos de todos los autores y editores:

```
for $l in distinct-values(doc("libros.xml")
      //(autor | editor)/apellido)
order by $l
return <apellidos>{ $l }</apellidos>
```

El resultado de esta consulta se muestra en el siguiente párrafo:

```
<apellidos>Abiteboul</apellidos>
<apellidos>Buneman</apellidos>
<apellidos>Gerbarg</apellidos>
<apellidos>Stevens</apellidos>
<apellidos>Suciu</apellidos>
```

Aplicación de funciones XQuery

El operador de intersección recibe dos secuencias de nodos como operandos y devuelve una secuencia conteniendo todos los nodos que aparezcan en ambos operandos.

El operador de sustracción (except) recibe dos secuencias de nodos como operandos y devuelve una secuencia conteniendo todos los nodos del primer operando que no aparezcan en el segundo operando. A continuación se muestra una consulta que usa el operador sustracción para obtener un nodo libro con todos sus nodos hijos salvo el nodo <precio>.

```
for $b in doc("libros.xml")//libro
where $b/titulo = "TCP/IP Illustrated"
return
<libro>
      { $b/@* }
      { $b/* except $b/precio }
</libro>
```

Aplicación de funciones XQuery

El resultado de esta consulta se muestra en el siguiente párrafo:

```
<libro year = "1994">
      <title>TCP/IP Illustrated</title>
      <author>
       <apellidos>Stevens</apellidos>
       <first>W.</first>
      </author>
      <publisher>Addison-Wesley</publisher>
</libro>
```

Aplicación de funciones XQuery

La función distinct-values() extrae los valores de una secuencia de nodos y crea una nueva secuencia con valores únicos, eliminando los nodos duplicados. Por ejemplo, la siguiente consulta devuelve todos los apellidos distintos de los autores.

```
for $l in distinct-values(doc("libros.xml")//autor/apellidos)
return <apellidos>{ $l }</apellidos>
```

El resultado de esta consulta se muestra en el siguiente párrafo:

```
<apellidos>Stevens</apellidos>
<apellidos>Abiteboul</apellidos>
<apellidos>Buneman</apellidos>
<apellidos>Suciu</apellidos>
```

Documento XML para las consultas:

```
<pelicula fechaEstreno="19/12/2007">
   <titulo>Los Otros </titulo>
   <director>Alejandro  Amenabar</director>
</pelicula>
<pelicula fechaEstreno="15/11/1986">
   <titulo>Matrix </titulo>
   <director>Ethan Coen </director>
</pelicula>
<pelicula fechaEstreno="14/06/1990">
   <titulo>No es país para pobres </titulo>
   <director>Ethan Coen </director>
   <director>Joel Coen </director>
</pelicula>
<pelicula fechaEstreno="19/12/2007">
   <titulo> Harry Potter </titulo>
   <director> Peter Jackson</director>
</pelicula>
```

Algunos Ejemplos de Consultas XQuery:

FOR: cada nodo recuperado por la expresión, es vinculado a una única tupla.

```
FOR $b IN doc("informacion.xml")//película/director
RETURN
<directores>$b</directores>
```

Recuperaría:

```
<directores>
  <director>Peter Jackson</director>
</directores>
<directores>
  <director>Alejandro Amenabar</director>
</directores>
<directores>
  <director>Ethan Coen</director>
</directores>
<directores>
  <director>Joel Coen</director>
</directores>
```

Algunos Ejemplos de Consultas XQuery:

LET: recupera todos los nodos en una única tupla.

```
LET $b IN doc("informacion.xml")//película/director
RETURN
<directores>$b</directores>
```

Recuperaría:

```
<directores>
   <director>Peter Jackson</director>
   <director>Alejandro Amenabar</director>
   <director>Ethan Coen</director>
   <director>Joel Coen</director>
</directores>
```

Algunos Ejemplos de Consultas XQuery:

WHERE: filtra la información de las tuplas devueltas por FOR o LET, recuperando únicamente aquellas que cumplen una expresión.

```
LET $b IN doc("información.xml")//película/director
WHERE $b/../@fechaEstreno="19/12/2007"
RETURN
<directores>$b</directores>
```

Recuperaría:

Recuperaría, únicamente los directores que hubiesen estrenado película el día 19-12-2007 (Suponemos fechaEstreno es un atributo por ello ponemos @ delante, el /../ es para subir un nivel en el árbol XML donde está la información):

```
<directores>
   <director>Peter Jackson</director>
   <director>Alejandro Amenabar</director>
</directores>
```

Algunos Ejemplos de Consultas XQuery:

ORDER: organiza la información recuperada. Por defecto, organiza por orden ascendente según el tipo de datos.

```
FOR $b IN doc("información.xml")//película/director
ORDER BY $b/../titulo
RETURN
<directores>$b</directores>
```

Recuperaría los directores ordenador por el titulo de sus películas:

```
<directores>
   <director>Alejandro Amenabar</director>
</directores>
<directores>
   <director>Ethan Coen</director>
</directores>
<directores>
   <director>Joel Coen</director>
</directores>
<directores>
   <director>Peter Jackson</director>
</directores>
```

Algunos Ejemplos de Consultas XQuery:

RETURN: formatea la información de salida. Esto tiene muchas utilidades ya que puedes convertir la información recuperada a HTML, PDF, o cualquier otro.

En los anteriores ejemplos ya utilizábamos la clausula RETURN para que nos devolviera los resultados entre los tags <directores>.

XQuery también admite expresiones condicionales IF-THEN-ELSE. Esta cláusula es útil para dar un formato diferente a la salida dependiendo de la información de esta, es decir, podemos cambiar la estructura de los nodos en los que recuperamos la información según nos convenga por su contenido. El ELSE es obligatorio pero puede ser únicamente ELSE ().

Algunos Ejemplos de Consultas XQuery:

Si tuvieramos: FOR $b IN doc("información.xml")//película/director
RETURN

Entonces:

```
IF ($b = "Alejandro Amenabar") THEN
<espanol>{ $b }</espanol>
ELSE
<directores>{ $b }</directores>
```

Recuperaría, todos los nodos <director> de todas las películas del documento:

```
<directores>
   <director>Peter Jackson</director>
</directores>
<directores>
   <director>Alejandro Amenabar</director>
</directores>
<directores>
   <director>Ethan Coen</director>
</directores>
```

Algunos Ejemplos de Consultas XQuery:

Cuantificadores existenciales

XQuery dispone de dos cuantificadores existenciales EVERY: recupera aquellas tuplas en las que todos los nodos cumplan la condición. SOME: recupera aquellas tuplas en las que algún nodo cumpla la condición.

Ejemplo:

```
LET $b IN doc("información.xml")//película/titulo
WHERE SOME $a in $b/../director
SATISFIES ($a = "Ethan Coen")
RETURN
<titulos>$b</titulos>
```

Recuperaría los títulos de las películas en las que algún director fuera "Ethan Coen"

```
<titulos>
   <titulo>Matrix</titulo>
   <titulo>No es país para pobres</titulo>
</ titulos >
```

Algunos Ejemplos de Consultas XQuery:

Cuantificadores existenciales

Igual pero con EVERY:

```
LET $b IN doc("información.xml")//película/titulo
WHERE EVERY $a in $b/../director
SATISFIES ($a = "Ethan Coen")
RETURN
<titulos>$b</titulos>
```

Recuperaría los títulos de las películas en las que todos los directores (en este caso el único) fueran "Ethan Coen"

```
<titulos>
   <titulo>Matrix</titulo>
</ titulos >
```